Chapter 14: Indexing

1 Q&A Examples

Question 1

What is the difference between **primary key** and **unique key** in databases?

Answer

- **Primary Key**: Ensures unique identification of each record and does not allow `NULL`.
- **Unique Key**: Enforces uniqueness, but allows a single `NULL` value.

Question 2

Explain the difference between **Dense Index** and **Sparse Index**.

Answer

- **Dense Index**: Every search key value has an entry in the index. *Pro: Fast lookup, Con: Larger index size.*
- **Sparse Index**: Only some search key values are indexed. *Pro: Smaller index size, Con: Requires extra block access in search.*

Question 3

Suppose we build a dense index on a relation with 1,000,000 tuples. Each index entry is small, so assume 100 entries fit in a 4KB block.

(a) How large will the index be if the relation has (i) 1,000,000 tuples, (ii) 100,000,000 tuples?

(b) How many random block reads are required to perform a binary search on an index of 10,000 blocks?

(c) Why is binary search on a large index costly, and how can a multilevel index

reduce the cost?

Answer

(a) Index size calculation:

- For 1,000,000 tuples:

$$\frac{1{,}000{,}000}{100} = 10{,}000 \text{ blocks } (\approx 40 \text{ MB}).$$

- For 100,000,000 tuples:

$$\frac{100{,}000{,}000}{100} = 1{,}000{,}000 \text{ blocks } (\approx 4 \text{ GB}).$$

(b) Binary search cost:

$$\lceil \log_2(10{,}000) \rceil = 14 \text{ random block reads.}$$

If one random read takes 10 ms, total time $= 14 \times 10\,\text{ms} = 140\,\text{ms}$.

(c) Why costly and how multilevel indices help:

- Binary search requires random I/O, which is expensive.
- **Solution: Multilevel index.** Build an outer index on the inner index.
- Example: For 10,000 blocks, outer index = 100 blocks ($\approx$ 400 KB).
- Outer index fits in memory $\rightarrow$ only 1 random I/O needed.

Question 4

How are **point queries** and **range queries** processed using a B^+-Tree index?

Answer

- **Point Query:** Traverse from the root to a leaf using key comparisons. Return pointer if key exists.
- **Range Query:** Traverse to lower bound leaf, then follow leaf links to collect all keys in range.
- **Efficiency:** Point queries $O(\log_{n/2} N)$ I/Os; range queries benefit from sequential leaf linkage.

Question 5

What is the **cost of query processing** in a B^+-Tree, and how does it compare with a binary search tree?

Answer

- **B^+-Tree:** Height $\approx \lceil \log_{\lceil n/2 \rceil}(N) \rceil$; typically 3–4 block reads.
- **Binary Tree:** Height $\approx \lceil \log_2(N) \rceil$; ≈ 20 random block reads for 1,000,000 records.
- **Conclusion:** B^+-trees are shallow and fat $\rightarrow$ faster for disk-based queries.

Question 6

How do B^+-Trees handle **non-unique keys** in an index?

Answer

- Use a **composite key** = (attribute, primary key) to avoid duplicate keys.
- Range queries can retrieve all matching records:

$$\texttt{findRange}((v, -\infty), (v, +\infty))$$

Question 7

Indices speed query processing, but it is usually a bad idea to create indices on every attribute, and every combination of attributes, that are potential search keys. Explain why.

Answer

- **Update Overhead:** Each index must be updated whenever the underlying table is modified (insert, update, delete), which slows down write operations.
- **Storage Cost:** Each index consumes additional disk space; creating too many indices can significantly increase storage requirements.
- **Diminishing Returns:** Many indices may never be used in queries, providing little benefit but adding maintenance overhead.
- **Complexity:** Managing many indices complicates database administration and may affect query optimizer decisions.

Question 8

Is it possible in general to have two clustering indices on the same relation for different search keys? Explain your answer.

Answer

- **No, generally it is not possible.**
- A **clustering index** determines the physical order of records on disk.
- Since records can be physically ordered in only one way, only **one clustering index per relation** is allowed.
- Secondary (non-clustering) indices can exist on other attributes, but they do not affect physical order.

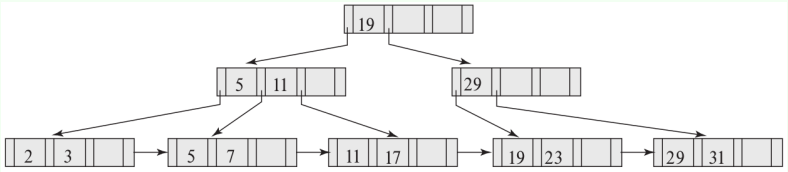
Question 9

Construct B^+-tree diagrams for the keys: $(2, 3, 5, 7, 11, 17, 19, 23, 29, 31)$ for three cases: (a) 4 pointers per node, (b) 6 pointers per node, (c) 8 pointers per node, with leaf links and leaf numbering.

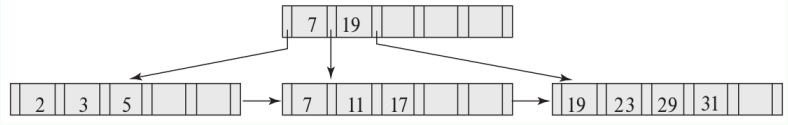
Answer

B^+-Tree Diagrams with Leaf Numbering:

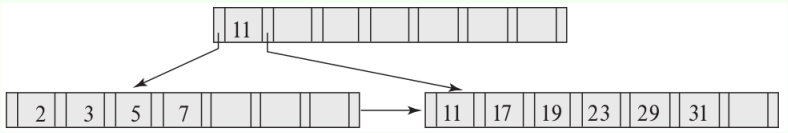
Case (a): 4 pointers per node (max 3 keys per node)



Case (b): 6 pointers per node (max 5 keys per node)



Case (c): 8 pointers per node (max 7 keys per node)



Note: Each leaf node is labeled $[L1], [L2], \ldots$ and connected sequentially with arrows to represent the linked list of leaves for efficient range queries.

Question 10

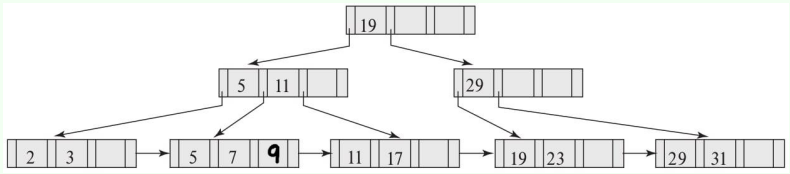
For each B^+-tree of Exercise 9, show the form of the tree after performing the following series of operations: (a) Insert 9 (b) Insert 10 (c) Insert 8 (d) Delete 23 (e) Delete 19

Answer

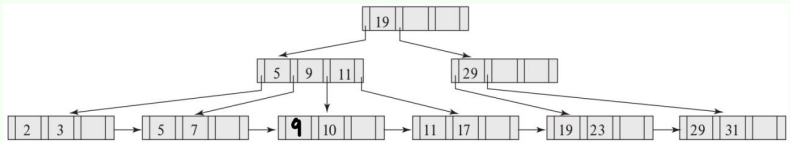
B^+-Tree Update Operations (Insertions and Deletions): The following diagrams illustrate how the B^+-tree evolves after each insertion and deletion. Each step specifies the operation performed and the resulting tree structure.

Case (a): 4 pointers per node (max 3 keys per node)

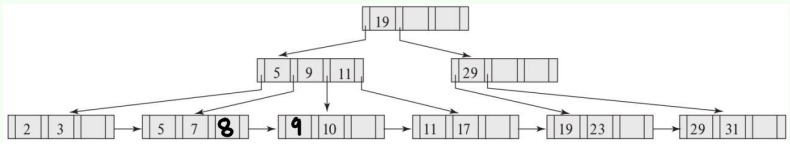
Step (a) — Insert 9:



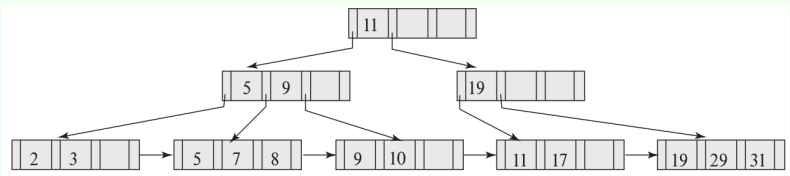
Step (b) — Insert 10:



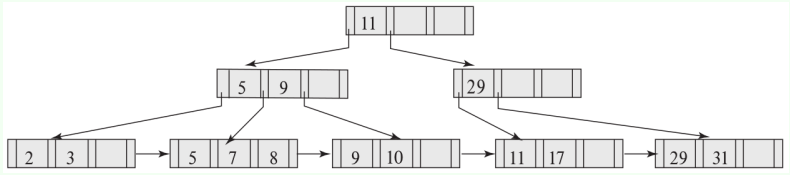
Step (c) — Insert 8:



Step (d) — Delete 23:

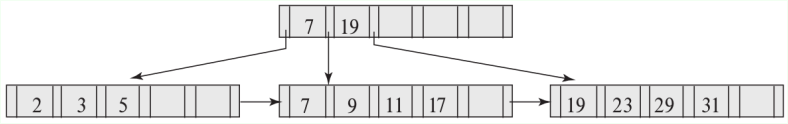


Step (e) — Delete 19:

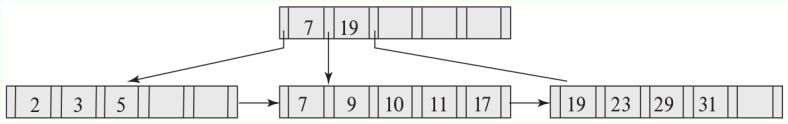


Case (b): 6 pointers per node (max 5 keys per node)

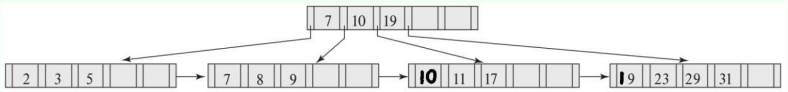
Step (a) — Insert 9:



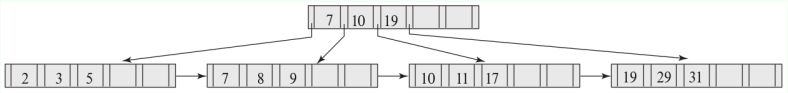
Step (b) — Insert 10:



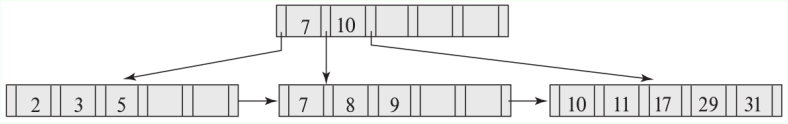
Step (c) — Insert 8:



Step (d) — Delete 23:

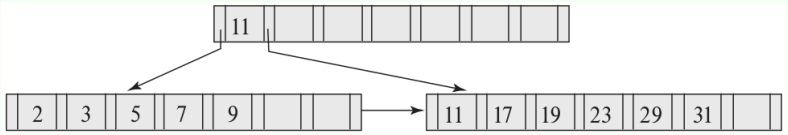


Step (e) — Delete 19:

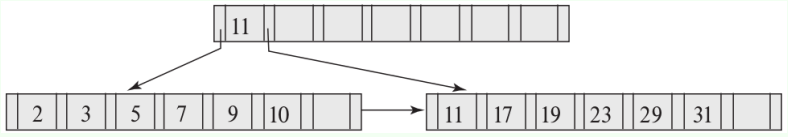


Case (c): 8 pointers per node (max 7 keys per node)

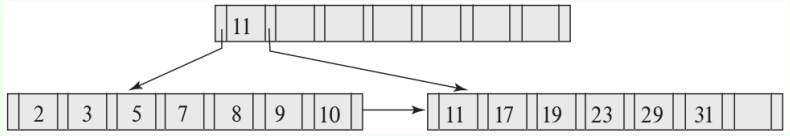
Step (a) — Insert 9:



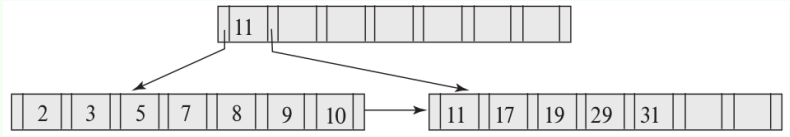
Step (b) — Insert 10:



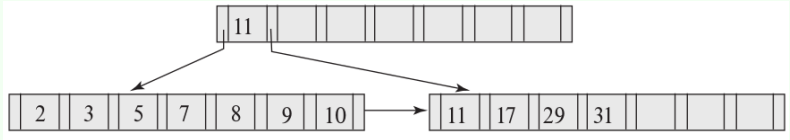
Step (c) — Insert 8:



Step (d) — Delete 23:



Step (e) — Delete 19:



Note: Each step shows the structural changes to the B$^+$-tree caused by insertion or deletion. Leaf nodes remain linked for sequential access, and leaf numbering continues as in Exercise 9.

Question 11

Consider the modified redistribution scheme for B$^+$-trees described, where up to m adjacent nodes may be involved in redistribution so that each node is guaranteed to contain at least

$$\left\lfloor \frac{(m-1)n}{m} \right\rfloor$$

entries, with n being the maximum number of entries per node. What is the expected height h of a B$^+$-tree built under this scheme as a function of n and the total number of entries N?

Answer

Expected height of the modified B$^+$-tree:

Let

$$N = \text{total number of search key values (or records)},$$

$$n = \text{maximum number of entries per node.}$$

Under the modified redistribution involving up to m adjacent nodes, each node is

guaranteed to contain at least

$$\left\lfloor \frac{(m-1)n}{m} \right\rfloor$$

entries. We therefore approximate the average branching factor B (number of children of a nonleaf) by

$$B \approx \beta\, n, \qquad \text{where} \qquad \beta = \frac{m-1}{m}.$$

The height h of the tree satisfies (approximately)

$$B^h \approx N,$$

so

$$h \approx \log_B N \;=\; \frac{\ln N}{\ln B} \;\approx\; \frac{\ln N}{\ln(\beta n)}.$$

Thus a convenient boxed form is

$$\boxed{h(N,n) \;\approx\; \frac{\ln N}{\ln\!\left(\dfrac{m-1}{m}\, n\right)}.}$$

Equivalently, for asymptotic reasoning,

$$\boxed{h(N,n) = \Theta\!\left(\frac{\log N}{\log n}\right)}$$

since $\ln(\beta n) = \ln n + O(1)$ for constant $\beta \in (0,1)$.

Remarks / special cases:

- Standard redistribution ($m = 2$): $\beta = \frac{1}{2}$, so $h \approx \log_{n/2} N$ (the usual B$^+$-tree estimate).
- Three-node scheme ($m = 3$): $\beta = \frac{2}{3}$, so $h \approx \log_{2n/3} N$, slightly smaller than the $m = 2$ case because nodes are on average more full.
- Increasing m improves node utilization (reduces height constant) but does not change the asymptotic dependence: height remains $\Theta(\log N / \log n)$.

Question 12

What would be the occupancy of each leaf node of a B$^+$-tree if index entries were inserted in sorted order? Explain why.

Answer

Explanation:

- When index entries are inserted in **ascending order**, each new key is directed to the **last (rightmost) leaf node**.
- Once this leaf node becomes full, it is **split into two**:
  - The **left leaf** retains roughly half of the entries.
  - The **right leaf** receives the new entry and becomes the target for future insertions.
- After the split, the left leaf remains untouched, and only the rightmost leaf continues receiving new entries.
- Consequently, most leaves stabilize at about **50% occupancy**, while only the final leaf (currently filling) may have higher occupancy.

Descending order insertions:

- The situation is symmetrical.
- All insertions go to the **first (leftmost) leaf node**.
- After each split, the **right leaf** remains untouched, and the **left leaf** continues to receive new entries.
- Hence, occupancy again remains around **50%** for all leaves except the first one.

Conclusion:

- Sequential (sorted) insertions cause poor space utilization in B^+-trees because each split leaves one node half full.
- To achieve better utilization, **bulk loading techniques** or **balanced insertion orders** should be used instead of inserting sorted data sequentially.

Question 13

Suppose you have a relation r with n_r tuples on which a secondary B^+-tree is to be constructed.

(a) Give a formula for the cost of building the B^+-tree index by inserting one

record at a time. Assume each block will hold an average of f entries and that all levels of the tree above the leaf are in memory.

(b) Assuming a random disk access takes 10 milliseconds, what is the cost of index construction on a relation with 10 million records?

Answer

Notation and assumptions:

$$n_r = \text{number of tuples (entries) to index,}$$

$$f = \text{average \# of entries per leaf block (leaf capacity),}$$

$$n = \text{max \# of keys (or pointers) per nonleaf node (node capacity).}$$

All nonleaf levels are kept in memory, so each insert requires accessing only the appropriate leaf block on disk. We ignore small constant bookkeeping costs and assume splits/redistributions add only a lower-order overhead (we show how to account for them).

(a) Cost formula for one-by-one insertion

For each insertion we must:

- read the target leaf block into memory (one random I/O), and
- write the modified leaf block back to disk (one random I/O).

Thus a typical insertion costs ≈ 2 disk I/Os. Splits cause additional I/Os (reading/writing a second leaf and updating parent nodes), but these are amortized: each split occurs roughly once per f insertions at a leaf. Including split overhead gives a small extra term.

Therefore the total number of random I/Os for building the index by single inserts is approximately

$$\text{I/Os} \;\approx\; 2\,n_r \;+\; O\!\left(\frac{n_r}{f}\right).$$

(The $O(n_r/f)$ term accounts for extra I/Os due to leaf splits and the parent updates they force.)

If t_{io} is the time per random I/O, the wall-clock cost is

$$\text{Time} \;\approx\; t_{io}\,\big(2\,n_r + O(n_r/f)\big).$$

(b) Numerical estimate for $n_r = 10{,}000{,}000$ and $t_{io} = 10$ ms

Using the leading term $2n_r$ (and ignoring the smaller split term for a conservative estimate):

$$\text{I/Os} \approx 2 \times 10{,}000{,}000 = 20{,}000{,}000 \text{ random I/Os.}$$

At 10 ms per random access:

$$\text{Time} \approx 20{,}000{,}000 \times 0.01 \text{ seconds} = 200{,}000 \text{ seconds.}$$

Convert:

$$200{,}000 \text{ s} \approx 55.56 \text{ hours} \approx 2.3 \text{ days.}$$

Even if splits and parent updates add, say, another 10% overhead, the time remains on the order of days. This illustrates why one-by-one insertion is impractical for very large relations and motivates bulk-loading.

Question 14

When is it preferable to use a dense index rather than a sparse index?

Answer

A **dense index** is preferable when:

- The data file is **unsorted (unordered)**, since every record needs a direct index entry.
- Queries frequently access **individual records** rather than ranges.
- The file is **small or frequently updated**, making per-record indexing manageable.

In contrast, a **sparse index** is suitable only for **sorted files** where entries correspond to block boundaries.

Question 15

What is the difference between a clustering index and a secondary index?

Answer

- **Clustering Index (Primary Index):** The search key defines the *sequential order* of the file. Only one clustering index can exist per relation.
- **Secondary Index (Nonclustering Index):** The search key defines an order *different* from the file's sequential order. Multiple secondary indices

can exist.

Question 16

Construct a B^+-tree for the following set of key values: $(2, 3, 5, 7, 11, 17, 19, 23, 29, 31)$.
For this B^+-tree, show the steps involved in the following queries:

(a) Find records with a search-key value of 11.

(b) Find records with a search-key value between 7 and 17, inclusive.

Answer

Step-by-Step Queries on the B^+-Tree:

(a) Search for key 11:

- Start at the **root** of the B^+-tree.
- Compare 11 with the keys in the root to select the correct child pointer.
- Traverse down internal nodes until reaching the **leaf node** containing 11.
- Return the record(s) corresponding to key 11.

(b) Search for keys 7 to 17:

- Start at the **root** and traverse to the leaf node containing 7.
- Follow the **leaf-level linked list** to collect all keys sequentially: 7, 11, 17.
- Stop when reaching the leaf node containing a key greater than 17.
- Return all records with keys in the range 7–17.

Note: The leaf nodes in a B^+-tree are linked, which allows efficient range queries without traversing back to the root.

Question 17

The solution presented in Section 14.3.5 to deal with nonunique search keys added an extra attribute to the search key. What effect could this change have on the height of the B^+-tree?

Answer

Adding an extra attribute (a uniquifier) to the search key increases the size of each key.

- Larger keys ⇒ fewer keys fit per internal node ⇒ decreased **fanout**.
- Decreased fanout ⇒ more levels needed to accommodate all entries ⇒ potential **increase in tree height**.

Question 18

Suppose there is a relation $r(A, B, C)$, with a B^+-tree index on the search key (A, B).

(a) What is the worst-case cost of finding records satisfying $10 < A < 50$ using this index, in terms of the number of records retrieved n_1 and the height h of the tree?

(b) What is the worst-case cost of finding records satisfying $10 < A < 50 \wedge 5 < B < 10$ using this index, in terms of the number of records n_2 that satisfy the selection, as well as n_1 and h defined above?

(c) Under what conditions on n_1 and n_2 would the index be an efficient way of finding records satisfying $10 < A < 50 \wedge 5 < B < 10$?

Answer

(a) Cost for $10 < A < 50$:

$$O(h + n_1)$$

where h is the height of the B^+-tree and n_1 is the number of records retrieved.

(b) Cost for $10 < A < 50 \wedge 5 < B < 10$:

$$O(h + n_1)$$

since we still need to traverse the tree to find the range on A (cost h) and then filter the n_1 records based on B to obtain n_2 records.

(c) Efficiency condition: The index is efficient if:

$$n_2 \approx n_1$$

That is, most of the records in the A-range also satisfy the B-range condition,

so few records are wasted in the filtering step.

Question 19

Suppose you have to create a B^+-tree index on a large number of names, where the maximum size of a name may be large (e.g., 40 characters) and the average name is itself large (e.g., 10 characters). Explain how prefix compression can be used to maximize the average fanout of nonleaf nodes.

Answer

Prefix Compression:

- Instead of storing the full search key in nonleaf nodes, store only a **prefix** sufficient to distinguish between the keys in the subtrees it separates.
- **Example:** If a nonleaf node separates subtrees containing "Silas" and "Silver", storing the prefix "Silb" is enough instead of the full "Silberschatz".
- **Effect:** Smaller key size in nonleaf nodes $\Rightarrow$ more keys fit per node $\Rightarrow$ **higher average fanout** and reduced tree height.

Question 20

Suppose a relation is stored in a B^+-tree file organization, and secondary indices store record identifiers that are pointers to records on disk.
Answer the following:

(a) What would be the effect on the secondary indices if a node split happened in the file organization?

(b) What would be the cost of updating all affected records in a secondary index?

(c) How does using the search key of the file organization as a logical record identifier solve this problem?

(d) What is the extra cost due to the use of such logical record identifiers?

Answer

(a) **Effect of node split:** When a node splits in the file organization, pointers in the secondary index that reference records in the split node must be updated to reflect the new locations.

(b) Cost of updating secondary index: Let h be the height of the secondary index B$^+$-tree and n be the number of affected records.

$$\text{Cost} = O(h \cdot n)$$

(c) Using logical record identifiers: If the search key of the file organization is used as the logical record identifier, secondary indices do not store physical pointers.

- Node splits in the file organization no longer require updates in the secondary index.
- This is a classic example of indirection: *"Every problem in Computer Science can be solved by another level of indirection."*

(d) Extra cost due to logical identifiers: Let h_1 be the height of the secondary index and h_2 be the height of the file organization.

- Using physical pointers: $O(h_1)$
- Using logical identifiers (search keys): $O(h_1 + h_2)$
- The extra cost comes from the need to traverse the B$^+$-tree of the file organization to locate the actual record.

Question 21

What trade-offs do write-optimized indices pose as compared to B$^+$-tree indices?

Answer

Trade-offs:

- **Pros (Write-Optimized Indices):**
  - Faster insertions, updates, and deletions due to batched or buffered writes.
  - Reduced I/O overhead for bulk modifications.
- **Cons:**
  - Queries can be slower, especially point lookups, because recent updates may reside in buffers or logs.
  - Increased complexity for maintaining consistency and merging updates into the main tree.

- **Summary:** Write-optimized indices trade query speed for faster write performance.

Question 22

Consider a hash table with 9 slots and the hash function $h(k) = k \bmod 9$. Demonstrate what happens upon inserting the keys 5, 28, 19, 15, 20, 33, 12, 17, 10 with collisions resolved by **chaining**.

Answer

Hash table (slots 0–8) using chaining:

- $h(5) = 5 \rightarrow$ insert 5 in slot 5.
- $h(28) = 1 \rightarrow$ insert 28 in slot 1.
- $h(19) = 1 \rightarrow$ insert 19 in slot 1 (collision, add to chain).
- $h(15) = 6 \rightarrow$ insert 15 in slot 6.
- $h(20) = 2 \rightarrow$ insert 20 in slot 2.
- $h(33) = 6 \rightarrow$ insert 33 in slot 6 (collision, add to chain).
- $h(12) = 3 \rightarrow$ insert 12 in slot 3.
- $h(17) = 8 \rightarrow$ insert 17 in slot 8.
- $h(10) = 1 \rightarrow$ insert 10 in slot 1 (collision, add to chain).

Resulting table:

Slot	Chain
0	-
1	28 $\rightarrow$ 19 $\rightarrow$ 10
2	20
3	12
4	-
5	5
6	15 $\rightarrow$ 33
7	-
8	17

Question 23

Consider inserting the keys 10, 22, 31, 4, 15, 28, 17, 88, 59 into a hash table of length $m = 11$ using **open addressing**. Illustrate the result of inserting these keys using:

(a) Linear probing with $h(k, i) = (k + i) \bmod m$

(b) Double hashing with $h_1(k) = k \bmod m$, $h_2(k) = 1 + (k \bmod (m - 1))$, and $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$

Answer

(a) Linear probing: $h(k, i) = (k + i) \bmod 11$

Index	Key
0	22
1	88
2	-
3	-
4	4
5	15
6	28
7	17
8	59
9	31
10	10

(b) Double hashing: $h_1(k) = k \bmod 11$, $h_2(k) = 1 + (k \bmod 10)$

Index	Key
0	22
1	-
2	59
3	17
4	4
5	15
6	28
7	88
8	-
9	31
10	10

Explanation:

- **Linear probing** searches the next sequential slot when a collision occurs.
- **Double hashing** uses a second hash function to compute a step size, reducing clustering and distributing keys more uniformly.
- The final positions of keys are calculated step by step using the respective formulas for collision resolution.